L18 - PPO 与 GRPO 强化学习

> **"在试错中持续进化"**

本节目标

1. 理解 PO 算法的统一框架：策略项 × 优势项 - 正则项
2. 掌握 PPO 的核心机制：概率比、clip、优势函数
3. 理解 GRPO（Group Relative Policy Optimization）的原理
4. 对比 PPO vs GRPO vs DPO 的差异
5. 了解奖励模型、奖励稀疏、RLAIF 等概念
6. 阅读 MiniMind 的 `train_rl.py` 源码

前置知识

- L17 DPO 偏好优化的基本概念
- 强化学习基础概念：策略、奖励、优势函数
- KL 散度的含义
- 了解 On-Policy 与 Off-Policy 的区别

一、PO 算法的统一视角

1.1 什么是 PO 算法

PO（Policy Optimization，策略优化）算法是一大类用于优化语言模型策略的方法。无论是 PPO、GRPO、DPO 还是 CISPO，它们都可以统一到一个框架下：

$$\mathcal{L}_{PO} = \mathbb{E}[f(r_t) \cdot g(A_t) - h(KL_t)]$$

其中： - **策略项** $f(r_t)$：与概率比相关的函数，描述"策略更新的方向和幅度" - **优势项** $g(A_t)$：优势函数的变换，描述"这个动作有多好" - **正则项** $h(KL_t)$：KL 散度惩罚，防止策略跑太远

1.2 概率比（Importance Ratio）

所有 PO 算法的核心概念——概率比：

$$r_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{old}(a_t \mid s_t)}$$

- π_θ：当前策略（正在优化的模型）
- π_{old}：旧策略（采样数据时的模型）
- a_t：第 t 步的动作（即生成的 token）
- s_t：第 t 步的状态（即前面所有的 token）

直觉理解：$r_t > 1$ 意味着当前策略比旧策略更倾向于选择 a_t；$r_t < 1$ 则相反。

1.3 不同 PO 算法在统一框架中的位置

算法	策略项 $f(r_t)$	优势项 $g(A_t)$	正则项 $h(KL_t)$
PPO	$\min(r_t, \text{clip}(r_t))$	A_t (GAE)	$\beta \cdot KL$
GRPO	$\min(r_t, \text{clip}(r_t))$	$\tilde{A}_t$ (组内排名)	$\beta \cdot KL$
DPO	隐式	隐式偏好	隐式
CISPO	置信区间约束	A_t	自适应

二、PPO 详解

2.1 PPO 的目标

PPO（Proximal Policy Optimization）的目标是最大化期望奖励，同时限制每次策略更新的幅度，防止"步子太大扯到蛋"。

2.2 原始策略梯度的问题

最原始的策略梯度：

$$\mathcal{L}_{PG} = -\mathbb{E}[r_t \cdot A_t]$$

问题：如果某个 r_t 特别大（当前策略和旧策略差异很大），梯度会爆炸，导致训练不稳定。

2.3 PPO 的 Clip 机制

PPO 的核心创新就是 clip（截断）机制：

$$\mathcal{L}_{PPO} = -\mathbb{E}[\min(r_t \cdot A_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon) \cdot A_t)]$$

其中 ε 通常取 0.2。

分情况讨论：

当 $A_t > 0$（好动作）： - 我们希望增大 r_t（增加这个动作的概率） - 但 clip 将 r_t 截断在 $1 + \varepsilon$，防止增幅过大 - 取 min：如果 $r_t > 1 + \varepsilon$，使用截断后的值

当 $A_t < 0$（坏动作）： - 我们希望减小 r_t（降低这个动作的概率） - 但 clip 将 r_t 截断在 $1 - \varepsilon$，防止减幅过大 - 取 min：如果 $r_t < 1 - \varepsilon$，使用截断后的值

图示理解：

```
                clip 区间
            ┌──────────┐
            │          │
            │  安全区域  │
            │          │
────────────┼──────────┼────────
     1-ε     1     1+ε        r_t

在 [1-ε, 1+ε] 区间内正常更新
超出区间则截断梯度，不再鼓励继续偏离
```

2.4 优势函数 A_t

优势函数衡量"某个动作比平均水平好多少"：

$A_t = R_t - V(s_t)$

- R_t：从时间步 t 开始的累积奖励
- $V(s_t)$：状态 s_t 的价值估计（由 Critic 网络给出）

为什么需要 Critic 网络?

在 LLM 场景中，奖励模型只在生成完整回答后给出一个总分。但我们需要知道每个 token 的贡献。Critic 网络（也叫 Value Network）负责估计"在生成到第 t 个 token 时，预期最终能获得多少奖励"。

GAE（Generalized Advantage Estimation）

实践中使用 GAE 来计算更平滑的优势估计：

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}$$

其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 是 TD 误差。

2.5 PPO 的完整损失

PPO 的完整训练目标包含三个部分：

$\mathcal{L} = \mathcal{L}_{PPO} + c_1 \cdot \mathcal{L}_{VF} + c_2 \cdot \mathcal{L}_{entropy}$

- $\mathcal{L}_{PPO}$：策略损失（clip 版本）
- $\mathcal{L}_{VF}$：Critic 网络的价值函数损失（MSE）
- $\mathcal{L}_{entropy}$：熵奖励，鼓励探索

2.6 PPO 训练的完整流程

```
1. 用当前策略模型生成多条回答
2. 用奖励模型对每条回答打分
3. 用 Critic 网络估计每个 token 的状态价值
4. 计算优势函数 A_t（用 GAE）
5. 计算 PPO clip 损失
6. 更新策略模型和 Critic 网络
7. 回到步骤 1（On-Policy：每次更新后都要重新采样）
```

三、GRPO 详解

3.1 GRPO 的动机

PPO 需要一个和策略模型同等大小的 Critic 网络，这带来了额外的计算和内存开销。对于小模型来说尚可，但对于大模型（7B+），多维护一个 Critic 网络的代价很高。

GRPO（Group Relative Policy Optimization）是 DeepSeek 提出的方法，**完全不需要 Critic 网络**。

3.2 GRPO 的核心思想

GRPO 的关键洞见：不需要精确估计每个状态的价值，只需要知道**一组回答中哪个相对更好**。

对于同一个 prompt，生成 G 个回答 $\{y_1, y_2, ..., y_G\}$，用奖励模型给每个回答打分 $\{r_1, r_2, ..., r_G\}$，然后用组内标准化来计算优势：

$$\tilde{A}_i = \frac{r_i - \text{mean}(r_1, ..., r_G)}{\text{std}(r_1, ..., r_G)}$$

3.3 GRPO 的损失函数

$$\mathcal{L}_{GRPO} = -\frac{1}{G}\sum_{i=1}^{G}[\min(r_t^{(i)}\cdot\tilde{A}_i,\ \text{clip}(r_t^{(i)},1-\varepsilon,1+\varepsilon)\cdot\tilde{A}_i)] + \beta\cdot D_{KL}(\pi_\theta||\pi_{ref})$$

和 PPO 很像，但有两个关键区别： 1. **优势函数不同**：用组内相对排名代替 Critic 估计 2. **KL 正则**：直接对 KL 散度做惩罚，而不是用 clip 来隐式约束

3.4 GRPO vs PPO

维度	PPO	GRPO
Critic 网络	✅ 需要	❌ 不需要
优势计算	GAE + Critic	组内相对排名
内存开销	高（额外一个大模型）	低
采样方式	每个 prompt 一条回答	每个 prompt G 条回答
适用场景	通用 RL	更适合 LLM 场景

3.5 为什么 GRPO 更适合 LLM?

在传统 RL 中，Critic 网络可以利用环境的 Markov 性质做精确的价值估计。但在 LLM 生成场景中：

1. 奖励通常只在序列结束时给出（稀疏奖励）
2. 状态空间极大（所有可能的 token 序列）
3. Critic 网络很难精确估计中间状态的价值

GRPO 绕过了这些问题，直接用最终奖励的相对大小来指导优化。

四、RLAIF vs RLHF

4.1 什么是 RLAIF

RLAIF（RL from AI Feedback）用 AI 模型代替人类来提供反馈：

```
RLHF: 人类标注偏好 → 训练 RM → RL 训练
RLAIF: AI 模型打分  → 直接用分数做 RL 训练
```

4.2 MiniMind 的选择

MiniMind 使用 **InternLM2-1.8B-Reward** 作为奖励模型，属于 RLAIF 的范畴。选择这个模型的原因：

1. **1.8B 参数**：够小，能在单卡上运行
2. **专为奖励打分训练**：输出归一化的奖励分数
3. **开源可用**：可以直接从 HuggingFace 下载

五、奖励稀疏问题

5.1 什么是奖励稀疏

在 LLM 的 RL 训练中，奖励模型通常在**完整回答生成完毕后**才给出一个分数。对于一个 200 token 的回答：

```
token 1 → 无奖励
token 2 → 无奖励
...
token 199 → 无奖励
token 200 → 终于拿到奖励！（比如 0.7 分）
```

模型很难判断这 0.7 分归功于哪些 token、哪些 token 拖了后腿。

5.2 小模型的 RL 困境

对于 MiniMind 这样的小模型（64M 参数），RL 训练面临额外的困难：

1. **生成质量不稳定**：小模型在探索时可能生成大量低质量回答
2. **奖励信号噪声大**：小模型的回答方差大，奖励分数波动剧烈
3. **优化效率低**：大量探索浪费在低质量区域

5.3 缓解策略

- 使用 KL 惩罚，防止模型跑太远
- 使用 GRPO 的组内排名，降低绝对奖励值的影响
- 适当减小学习率
- 从一个好的 SFT checkpoint 开始

六、On-Policy 的挑战

6.1 On-Policy 意味着什么

On-Policy 方法（PPO/GRPO）每次更新策略后，旧数据就"过期"了，必须用新策略重新采样。

```
Epoch 1: 用 π_θ₀ 生成 → 计算 loss → 更新到 π_θ₁
Epoch 2: π_θ₀ 的数据作废 → 用 π_θ₁ 重新生成 → ...
```

6.2 计算开销分析

对于一次 PPO 迭代： 1. **生成阶段**：策略模型做推理，生成回答（慢！） 2. **评分阶段**：奖励模型打分 3. **训练阶段**：反向传播更新参数

其中生成阶段通常是最耗时的，因为自回归生成是 token-by-token 的。

6.3 与 DPO 的对比

DPO（Off-Policy）可以复用同一批数据训练多个 epoch，不需要重新生成。这使得 DPO 的训练效率远高于 PPO/GRPO。

七、MiniMind 源码解读：train_rl.py

7.1 整体流程

```
# 加载策略模型、参考模型、奖励模型
policy_model = load_model(checkpoint)
ref_model = load_model(checkpoint)
reward_model = load_reward_model("internlm2-1_8b-reward")

for epoch in range(num_epochs):
    for batch in dataloader:
        prompts = batch['prompts']

        # 1. 用策略模型采样 G 个回答
        responses = []
        for prompt in prompts:
            for _ in range(G):
                resp = policy_model.generate(prompt)
                responses.append(resp)

        # 2. 用奖励模型打分
        rewards = reward_model.score(prompts, responses)

        # 3. 计算组内标准化优势（GRPO 方式）
        advantages = group_normalize(rewards, G)
```

```
        # 4. 计算策略梯度损失 + KL 惩罚
        loss = compute_grpo_loss(policy_model, ref_model,
                                 prompts, responses,
                                 advantages, beta)

        # 5. 更新
        loss.backward()
        optimizer.step()
```

7.2 组内标准化

```
def group_normalize(rewards, group_size):
    """将每组 G 个回答的奖励做标准化"""
    rewards = rewards.view(-1, group_size)  # [B, G]
    mean = rewards.mean(dim=1, keepdim=True)
    std = rewards.std(dim=1, keepdim=True)
    advantages = (rewards - mean) / (std + 1e-8)
    return advantages.view(-1)  # [B*G]
```

7.3 KL 散度计算

```
def compute_kl(policy_logprobs, ref_logprobs):
    """逐 token 计算 KL 散度"""
    kl = policy_logprobs - ref_logprobs
    return kl.sum(dim=-1).mean()
```

八、CISPO 简介

MiniMind 还实现了 CISPO（Confidence Interval based Safe Policy Optimization）。与 PPO 的 clip 不同，CISPO 使用置信区间来约束策略更新：

- 根据采样数据的统计特性，计算策略更新的置信区间
- 只在置信区间内更新，超出部分自动收缩
- 理论上比 clip 更精细，实践中也有一定优势

🎤 面试考点

Q1: PPO 的 clip 机制是什么？为什么需要它？

答：PPO 将概率比 r_t 截断在 $[1-\varepsilon, 1+\varepsilon]$ 范围内，防止单次策略更新幅度过大。这是因为原始策略梯度中如果 r_t 特别大，梯度会爆炸导致训练不稳定。clip 机制相当于在策略空间中划了一

个"信任区域"，只允许在这个区域内更新。

Q2: GRPO 和 PPO 的主要区别是什么？

答： 1. GRPO 不需要 Critic 网络，而 PPO 需要 2. GRPO 用组内相对排名计算优势，PPO 用 GAE + Critic 3. GRPO 每个 prompt 生成多个回答做组内比较，PPO 通常每个 prompt 一个回答 4. GRPO 内存开销更低，更适合大模型 RL 训练

Q3: 什么是奖励稀疏？为什么它是个问题？

答：奖励稀疏指的是在 LLM 场景中，奖励模型只在完整回答生成后才给出一个整体分数，中间各个 token 没有即时奖励。这导致信用分配困难——模型不知道哪些 token 贡献了正奖励，哪些贡献了负奖励。

Q4: On-Policy 和 Off-Policy 的区别？各有什么优缺点？

答： - On-Policy（PPO/GRPO）：每次策略更新后必须用新策略重新采样数据。优点是训练信号准确，探索能力强；缺点是计算开销大，数据效率低。 - Off-Policy（DPO）：使用预先收集的固定数据集。优点是训练高效，数据可复用；缺点是不做在线探索，存在分布偏移。

Q5: PPO 中优势函数 A_t 是什么？为什么不直接用奖励？

答：优势函数 $A_t = R_t - V(s_t)$，表示某个动作比平均水平好多少。直接用奖励 R_t 会导致高方差（因为奖励的绝对值变化大），减去基线 $V(s_t)$ 后方差更低，训练更稳定。这就是方差缩减（variance reduction）技术。

Q6: MiniMind 使用什么作为奖励模型？为什么选它？

答：MiniMind 使用 InternLM2-1.8B-Reward 作为奖励模型。选择它因为：(1) 参数量小（1.8B），单卡可以运行；(2) 专门为奖励打分训练的；(3) 开源可用。这属于 RLAIF（AI 反馈）而非 RLHF（人类反馈）。

Q7: PPO 损失函数的完整形式是什么？

答：

$$\mathcal{L} = -\mathbb{E}[\min(r_t \cdot A_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon) \cdot A_t)] + c_1 \cdot \mathcal{L}_{VF} + c_2 \cdot \mathcal{L}_{entropy} + \beta \cdot D_{KL}$$

包含四个部分：策略 clip 损失、价值函数损失、熵奖励和 KL 惩罚。

✅ 自测题

1. **填空**：PPO 中 clip 的范围是 [___, ___]，ε 通常取 ___。
2. **判断**：GRPO 需要一个 Critic 网络来估计状态价值。（对/错）
3. **选择**：以下哪个不是 On-Policy 方法的特点?
4. A. 每次更新后需要重新采样
5. B. 训练数据可以反复使用
6. C. 计算开销较大
7. D. 探索能力较强
8. **简答**：GRPO 是如何计算优势函数的？与 PPO 有什么不同?
9. **计算**：如果一个 prompt 生成了 4 个回答，奖励分别是 [0.2, 0.8, 0.5, 0.3]，请计算 GRPO 的标准化优势。

▼ 参考答案

1. $[1-\varepsilon, 1+\varepsilon]$，ε 通常取 0.2 2. 错。GRPO 不需要 Critic 网络。 3. B。Off-Policy 才可以反复使用训练数据。 4. GRPO 对同一个 prompt 生成 G 个回答，用奖励模型打分后做组内标准化（减均值除标准差），得到相对优势。PPO 则使用 Critic 网络估计状态价值，通过 GAE 计算优势。 5. mean = 0.45, std = 0.2345 - 优势 = [(0.2-0.45)/0.2345, (0.8-0.45)/0.2345, (0.5-0.45)/0.2345, (0.3-0.45)/0.2345] - ≈ [-1.066, 1.493, 0.213, -0.640]

下一节预告

下一节我们将学习 **MoE 混合专家模型**，了解如何用多个专家网络来扩展模型容量，同时控制计算开销。MiniMind-3-MoE 用 4 个专家实现了 198M 总参数但仅 64M 激活参数的高效架构。